Use of the residue theorem in locating points within an arbitrary multiply-connected region

G. STEVEN GIPSON

Department of Civil Engineering, Louisiana State University, Baton Rouge, LA 70803, USA

A novel application of the residue theorem from complex variable theory is used to construct an algorithm that determines whether or not a given point lies within a two-dimensional region. The method applies to the most geometrically complex domains characterized by multiple connectivity and non-convexity. The usefulness of the technique is demonstrated with two examples of practical engineering interest.

Key Words: residue theorem, point-in-polygon, salt domes, Monte Carlo quadrature, program, boundary elements.

INTRODUCTION

Sloan[1] has recently presented a point-in-polygon algorithm based upon the 'enlarged orientation theorem' of Nordbeck and Rystedt.[2] The algorithm so derived is good for the determination of whether or not a point lies within a polygon of arbitrary shape. Such programs are of much importance in the efficient handling of many tactical problems in engineering analysis. Sloan[1] cites the example of mesh generation for finite element grids and the processing of real estate data. Other examples of the uses of such algorithms include imposition of complicated boundary conditions in finite element and boundary element data, and for integration over oddly shaped regions.

The purpose of this paper is to present an alternate technique for determining the status of a point with respect to whether it is inside or outside of a two-dimensional domain, and to illustrate some engineering applications of such a program. The new algorithm is based upon the residue theorem of complex variable theory and can be applied to any two-dimensional region whatsoever. We begin with a basic discussion of the residue theorem, its geometrical interpretation, and how it may be applied to a generalized point-in-polygon problem.

THE RESIDUE THEOREM

The residue theorem represents one of the most fundamental and useful applications of functions of a complex variable. The theorem is covered in detail in any textbook treatment of complex variables,[3] and therefore only a rudimentary discussion relative to our proposed application is given here.

We consider a function $f(z)$ where $z = x + iy$ is the usual independent complex variable. In a given region R bounded by a curve C, we restrict $f(z)$ to be in the class of analytic functions except for possibly at a finite number of points z_l. At these points, f may have either finite poles or essential singularities. The residue theorem states that

$$\int_C f(z)\,\mathrm{d}z = 2\pi i\,\Sigma(\text{Residues in } R) \tag{1}$$

The residue at the singularity z_l is defined by

$$\mathrm{Res}(f, z_l) = \frac{1}{2\pi i}\int_{C_l} f(z)\,\mathrm{d}z \tag{2}$$

where C_l is an anticlockwise oriented contour enclosing the pole at z_l and only that pole (Fig. 1).

There are a variety of ways to evaluate the residue. For our purposes, we shall exploit the result that occurs when the Laurent expansion is used for the residue evaluation. If by any method it is possible to find the coefficient a_{-1} in the Laurent expansion

$$f(z) = \sum_{n=-\infty}^{\infty} a_n(z - z_l)^n \tag{3}$$

then it can be shown that[3]

$$\mathrm{Res}(f, z_l) = a_{-1} \tag{4}$$

Of particular interest to us is the result that occurs when we take

$$f(z) = 1/(z - z_l) \tag{5}$$

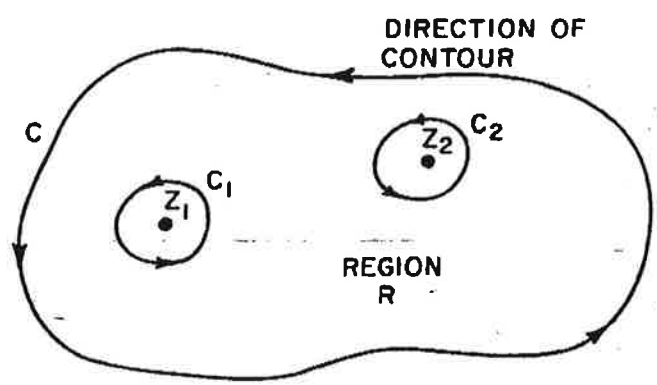


Figure 1. The idea behind the residue theorem. The result of integrating $f(z)$ around the contour C containing essential singularities or poles at z_1 and z_2 is the same as the sum of the integrals around C_1 and C_2

Accepted August 1985. Discussion closes June 1986.

0141-1195/86/020073-08$2.00
© 1986 Computational Mechanics Publications

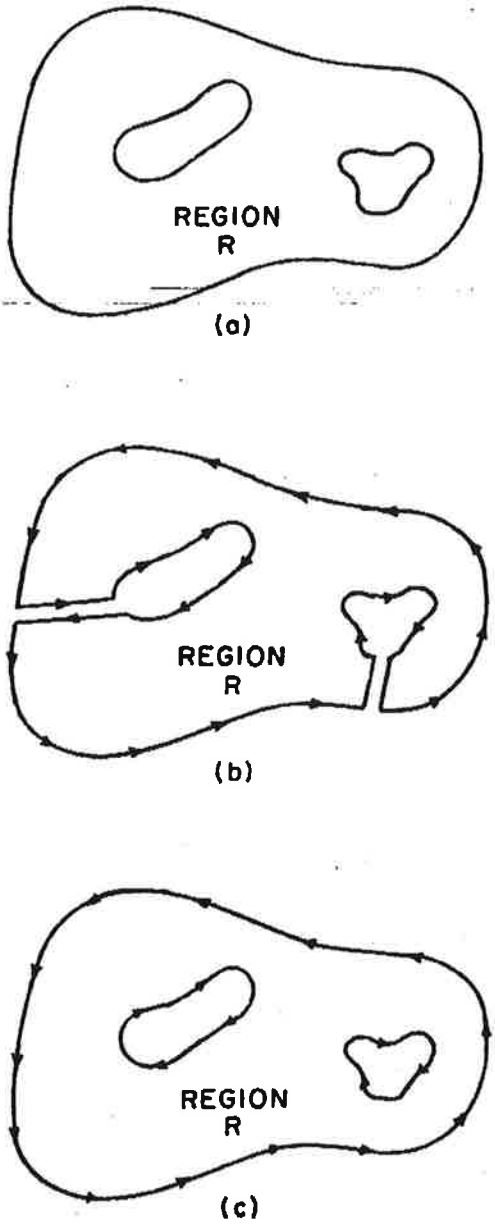


Figure 2. (a) A multiply connected region R may be handled with a single contour integral by (b) cutting through to each internal surface and exiting along the same path. This has the effect in (c) of integrating anticlockwise on external surfaces and clockwise on internal surfaces

In this case, it is seen very quickly from equations (3) and (4) that for any contour enclosing z_i:

$$\mathrm{Res}(f, z_i) = 1 \tag{6}$$

whereas if the contour does not enclose R, the residue vanishes. If we can guarantee that a simple pole such as that defined at z_i in equation (5) is the only singularity in the entire complex plane, then equations (5) and (6) suggest an excellent point-in-region testing algorithm. These results do in fact define the algorithm that is proposed in the next section of this work.

Before we proceed, however, we should address two other questions. The first has to do with the possibility Fig. 2(a). Because the use of the residue theorem involves performing a single contour integral, we must deform the contour as illustrated in Fig. 2(b) to traverse the entire boundary. The technique is to force the path taken inward to an 'island' to coincide with the path taken outward to get back to the external boundary. The line integrals cancel each other along this path. In this way, we find the result depicted in Fig. 2(c) which dictates that an internal boundary should be traversed in a clockwise direction for the residue theorem to hold for a singularity inside R.

The other question concerns what happens when the singularity lies on the contour. In this case, the point should be approached along the contour to within a distance ϵ and then bypassed on a circular arc of radius ϵ subtending an angle α before continuing along the contour as shown in Fig. 3. We then perform the line integral and take the limit as $\epsilon \to 0$. If the singularity is the first order pole $1/(z-z_i)$, then the entire contour integral may be written as the sum of the contour integral minus the ϵ-circle, plus the portion on the ϵ-circle, i.e.:

$$\int_C \frac{dz}{z-z_i} = \int_{C-\epsilon} \frac{dz}{z-z_i} + \int_{\epsilon} \frac{dz}{z-z_i} \tag{7}$$

Writing $z-z_i = R_0 \exp(i\theta)$, the last term in equation (7) becomes

$$\int_{\epsilon} \frac{dz}{z-z_i} = \int_0^{\alpha} \frac{R_0 i e^{i\theta}\, d\theta}{R_0 e^{i\theta}} = i \int_0^{\alpha} d\theta = i\alpha \tag{8}$$

Therefore, taking the limit in equation (7) yields

$$\int_C \frac{dz}{z-z_i} = i(2\pi - \alpha) \tag{9}$$

This result has an interesting geometrical interpretation if we take a less restrictive attitude toward the mathematical concept of a 'point'. What we shall do is consider the point p to occupy a vanishingly small circular region of space of circumference $2\pi\epsilon$. Equation (9) would seem to indicate that at any such 'point' where a $1/(z-z_i)$ type singularity exists, the residue is equal to the fraction of the 'point' contained within the contour times 2π. Interpreted in this rather loose manner, equation (9) represents a generalization of the basic residue theorem for this type of singularity. It also provides the means for devising a computational scheme by which any point may be tested as to its geometric location with respect to a planar region. The

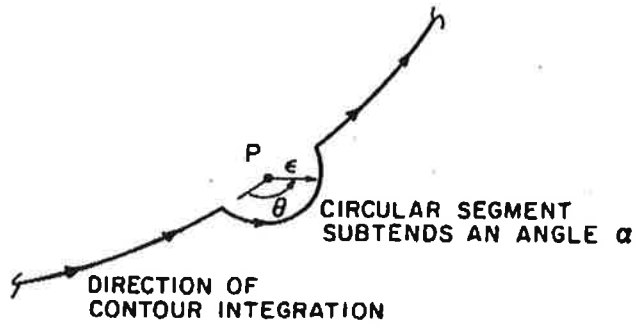


Figure 3. Handling of a point on the boundary by slightly

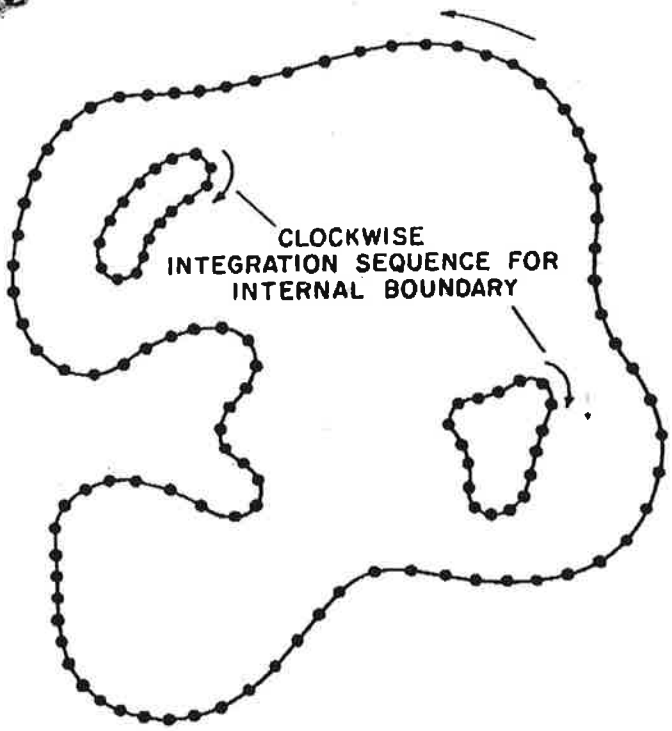


Figure 4. A contour defined by a sequence of nodal points indexed sequentially in anticlockwise order on external boundaries and in clockwise order on internal boundaries

application of the residue theorem for this purpose is described in the next section.

DESCRIPTION OF THE ALGORITHM

We consider an arbitrary two-dimensional region over which a sequence of points is to be tested as to whether they reside inside or outside a given region R. The region may contain any number of cusps, indentations, etc., and may be multiply connected to any order. It is assumed that the region has a boundary defined in some convenient manner. Normally, this would be accomplished by using either an explicit functional definition of the boundary or by defining a discrete set of surface points such as is done in boundary element analysis.[4] We will assume in the ensuing discussion that the latter definition of the boundary will be used because it provides a more general means of arbitrary boundary representation. A boundary grid of sufficient generality is shown in Fig. 4. The defining nodal points are indexed sequentially in anticlockwise fashion on the external boundary and clockwise on the internal boundary. Although any variety of spline curves or shape functions may be used to connect the points together, we will limit the discussion here to a case where any closed curve in the plane is a simple 'connect-the-dots' polygon; that is, the points will be connected together by simple straight line segments. However, the idea may be generalized easily to a spline of arbitrary complexity.

The use of the residue theorem proceeds in discrete fashion as shown in Fig. 5. A point P to be tested is introwritten as the sum of many integrals over the defining line segments. Placing the local origin of coordinates at z_p and writing $z-z_p = r\exp(i\theta)$, we find for the integral over segment e_j:

$$\int_{e_j} \frac{dz}{z-z_p} = \int_{e_j} \frac{[\exp(i\theta) + ir\exp(i\theta)]}{r\exp(i\theta)}\, d\theta$$

$$= \int_{e_j} \frac{dr}{r} + i\int_{e_j} d\theta \qquad (10)$$

The residue theorem states that only the imaginary part of the entire contour integral can be non-zero. Therefore, we ignore the first term in equation (10) and concern ourselves with

$$\mathrm{Im}\int_{e_j} \frac{dz}{z-z_p} = (\theta_2 - \theta_1) \qquad (11)$$

which is just what would be expected in light of the result obtained by performing a complete contour integral.

Equation (11) reduces the computational labor somewhat because it eliminates the need to perform an explicit integration. As suggested in Fig. 5, only the angle $\theta_2-\theta_1$ between the hashed lines connecting the nodal points needs to be computed. This is done effectively by simply using law of cosines. Each boundary segment is integrated over in

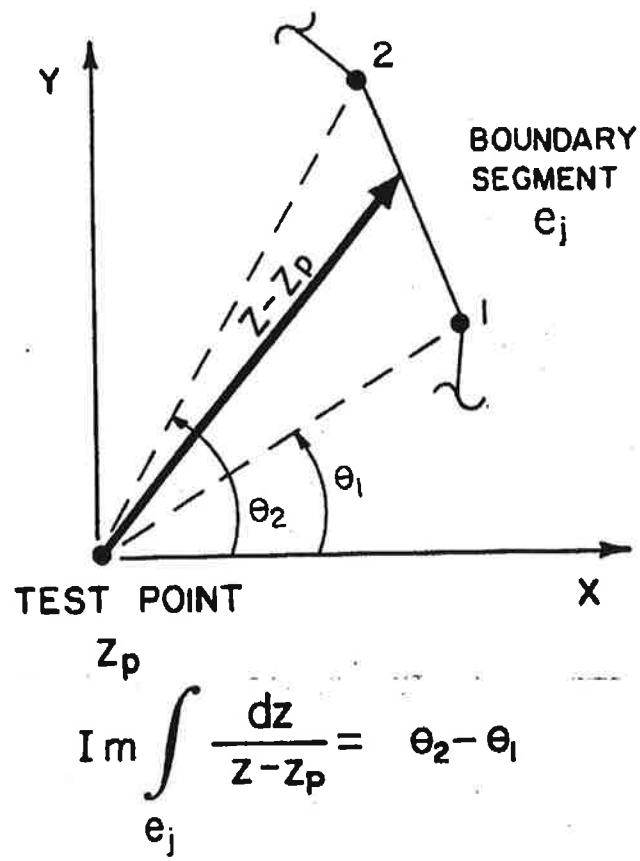


Figure 5. Schematic of an integration over a boundary segment e_j to test a point P located at global coordinate z_p

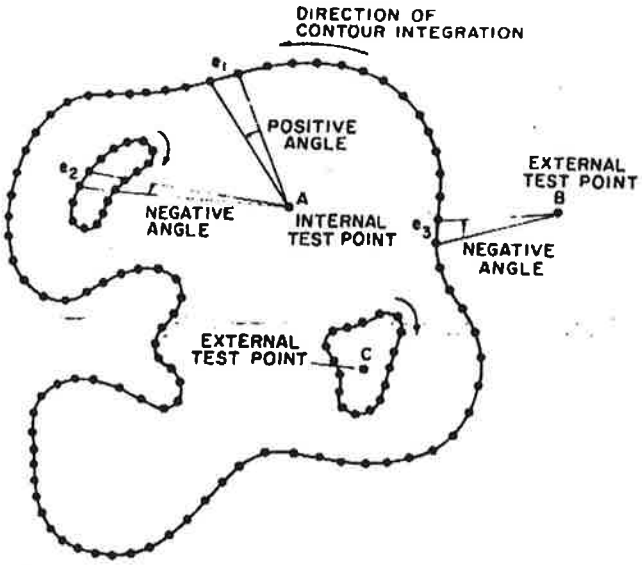


Figure 6. Various test points and sample integration segments

this manner and a running sum of the calculated angles is maintained. After all elements have been looped over, the sum for most points will either be 2π, indicating that the point is inside the region, or zero, indicating that the point is outside. The possibility that the point is on the boundary as discussed above will yield a total angle somewhere between these two extremes as prescribed by equation (9).

It is interesting to watch the way the sum develops pictorially as is illustrated in Fig. 6. An internal test point denoted by point A will yield a contour integration of 2π after the external boundary is looped over, the positive contributions to the sum upon entering a reentrant surface being necessarily negated at some point on the contour. The sum of the angles around the internal surfaces is obviously zero for such a point. For point B completely outside, the sums of the angles over each of the three boundaries are each identically zero. Point C, which is outside the region but completely surrounded by it, yields a sum of 2π for the anticlockwise contour, a sum of zero for the internal contour containing element e_2, and a sum of -2π for the sum around its immediate boundary. The total result is a zero angle and the point is declared external. The complexity of the region in Fig. 6 illustrates graphically the power of the algorithm in selecting internal points for a very general region. It should also be noted that this simple geometrical interpretation of the residue theorem involves essentially the same process as that prescribed by the theorem of Nordbeck and Rystedt.[2] Therefore, the equivalence of the two techniques is established.

Programming the algorithm according to these principles is extremely simple. A FORTRAN function subprogram named OUTIN designed on the preceding principles is presented in the Appendix. The code produces a value of OUTIN = 1.0 for points inside the region and OUTIN = 0.0 for points outside the region. Experience has shown that a tolerance of $2\pi \times 10^{-2}$ on the angle sums is sufficient for testing purposes.

None of the author's applications of the program have ever required the knowledge of test points on the boundary. However, the computational expense of testing for this contingency is negligible. Since this case may be important to some researchers, a feature has been installed into OUTIN where the program returns a value of OUTIN = 0.5 if the test point is on the boundary. The way this is done for linear boundary segments defining the region is particularly simple. OUTIN uses a cross product test to determine the sign of the angle between the two vectors connecting the endpoints of the boundary segment with the test point. (This is the angle $\theta_2 - \theta_1$ between the hashed lines in Fig. 5.) The cross product is identically zero only if the test point lies on the boundary segment. As OUTIN loops over the boundary segments summing angles, it checks for a zero cross product (within a tolerance of 10^{-6}) on each. If the test is true, it immediately returns control to the calling program with OUTIN = 0.5. This tolerance should be decreased in value if very small boundary segments or very large domains are being analyzed. As a guideline, the tolerance should be less than the length of the smallest boundary segment divided by twice the maximum distance across the domain.

FUNCTION OUTIN has been used successfully in many problems of engineering interest. Two of these applications are presented in the next section.

APPLICATIONS

Application of boundary condition in finite element analysis of salt domes

In finite element modeling of large tectonic structures, odd geometries often make the imposition of non-uniform boundary conditions difficult to apply. An example of this occurs in the three-dimensional stress/displacement analysis of salt domes. The situation is illustrated in Fig. 7 where a horizontal cross-section through a large 3-D finite element model of Vacherie salt dome in northern Louisiana, USA, is shown. The figure represents the bottom layer of nodes on the model. Inside the bold lines is the salt dome which has a roughly elliptical cross-section. Outside the dome to the edge of the mesh is earthy material. In order to simulate the uplift of the dome due to the effects of hydrostatic instability, it is required to apply appropriate displacement boundary conditions perpendicular to the plane of Fig. 7. It is assumed that the rocksalt behaves much like a viscous fluid flowing through a pipe as the dome gravitates upward through the earth strata over large periods of geologic time. The head of the flow field is noted by w_{max}. It is assumed to be located at the geometric center (x_c, y_c) of the dome cross-section. The vertical displacement boundary conditions may be summarized by the following equations:

$$w(x,y) = w_{max}\left[1 - \frac{(x-x_c)^2 + (y-y_c)^2}{R_B^2}\right] \text{for } (x,y) \in R \quad (12)$$

and

$$w(x,y) = 0 \text{ otherwise} \quad (13)$$

R_B is the distance between the point (x_c, y_c) and the boundary of the dome along the vector $(x-x_c, y-y_c)$ as shown in Fig. 8(a). The velocity profile appears somewhat like that shown in Fig. 8(b).

Without a suitable point-in-polygon program, applying boundary conditions of this sort would probably have to be done by manual calculation. With FUNCTION OUTIN, the problem was solved quite easily. First, the nodal points delineating the boundary of the salt dome region were tagged and organized in an anticlockwise sequence. A calling program which looped over the nodal coordinate data was written. The program tested each point to see if it was

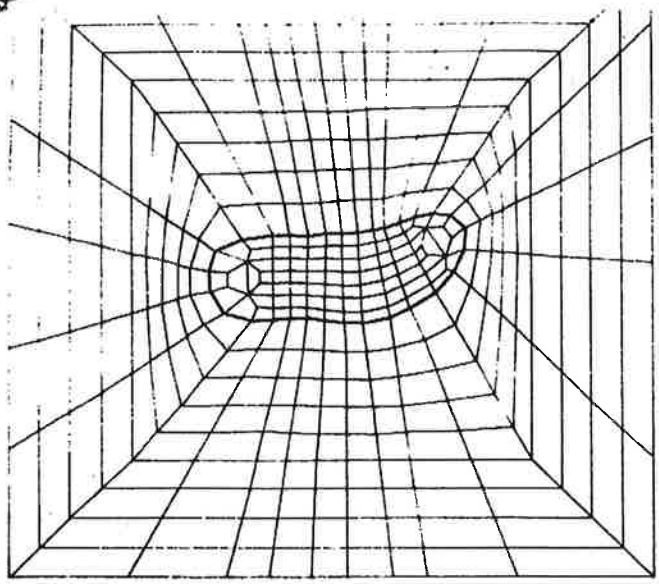
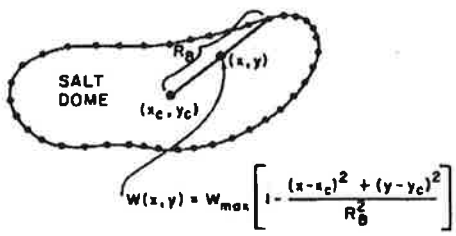
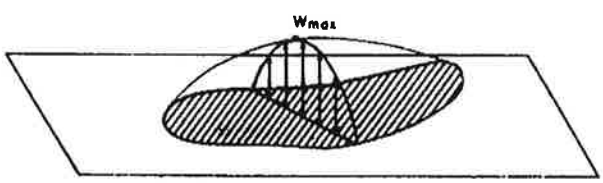


Figure 7. A horizontal layer of nodes in a three-dimensional finite element mesh of a salt dome. The salt dome is inside the bold lines



(a)



(b)

Figure 8. (a) The displacement function to be imposed at nodes located within the indicated contour. The resulting displacement field is a paraboloidal bulge in the plane containing the nodes

inside or outside the dome boundary and applied the appropriate boundary condition.

Monte Carlo quadrature over an irregular region[5-7]

One of the most useful applications of FUNCTION OUTIN is in performing general integrals over oddly shaped regions. In particular, the author has experienced much success in applying Monte Carlo quadrature (MCQ) to domain integrals containing singular integrands in the boundary element method.[5,6] The Monte Carlo methods is integrable. However, their accuracy depends upon finding a set of functional values distributed within the region of integration according to some convenient probability distributions.[7,8] For an arbitrary region, effecting a MCQ integration is an intractable problem without the aid of a routine like OUTIN. A brief description of the general ideas behind the MCQ technique and its implementation are given below.

The Monte Carlo quadrature methods (as distinct from Monte Carlo simulations of physical processes) are not well known but are actually quite old and the theory behind MCQ is well established. The precise details of the theory may be found elsewhere.[7,8] Here, we simply summarize the basic conceptual ideas behind MCQ.

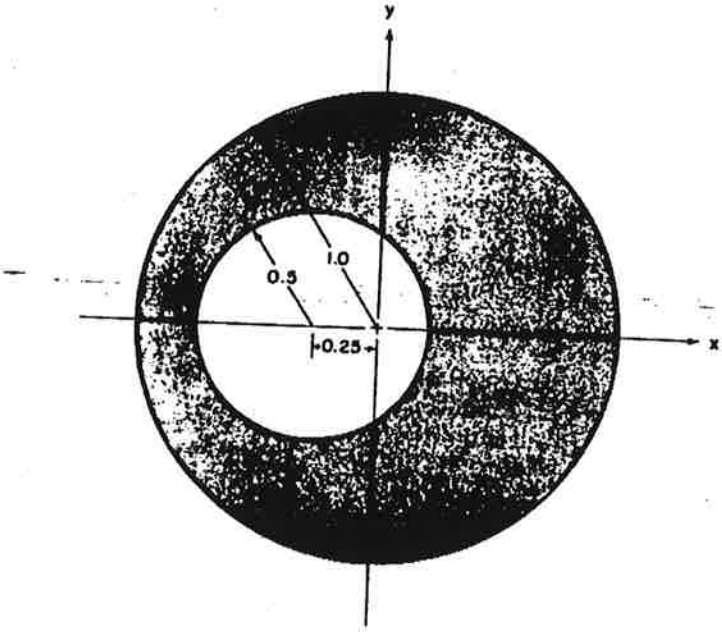
Let $f(x, y)$ be an arbitrary function to be integrated over a general two-dimensional domain R of the type described previously. Then at any point (x_i, y_i) chosen at random within R, we may calculate a value $f_i = f(x_i, y_i)$. If many such functional values are picked from a population of random numbers distributed with a probability density $P(x, y)$, then the expectation value of the integral we seek is:

$$I = \left[A \sum_{i=1}^{N} f(x_i, y_i)\right] \Big/ N \cong \iint_R f(x, y)\, P(x, y)\, \mathrm{d}x\, \mathrm{d}y \quad (14)$$

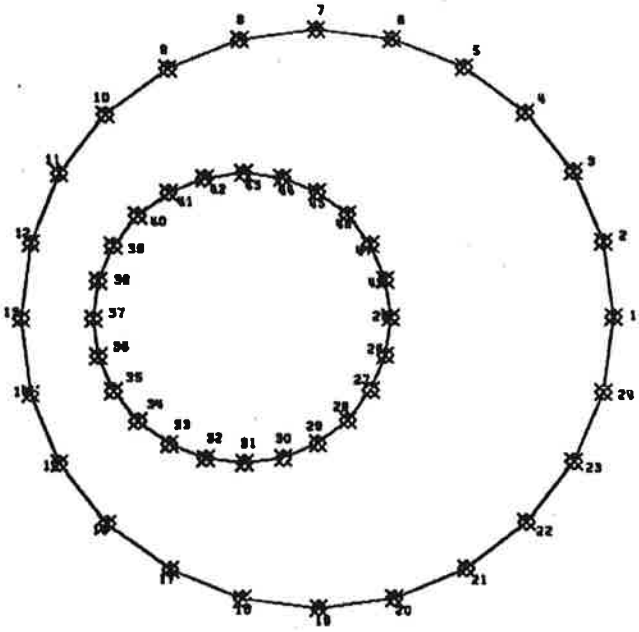
where N is the total number of random points chosen and A is the total area of the region R. Inspection of equation (14) shows that the mechanics of computing the integral is not unlike the procedure for computing the centroid of an area by composite sections. The result for I becomes exact as N goes to infinity. It can be shown that the error in the approximation is directly related to the factor $1/\sqrt{N}$ for N taken sufficiently large.[7] All that is required to implement the simple summation scheme suggested by equation (14) is a pseudo-random number generator with a known probability distribution and SUBROUTINE OUTIN. The success of the algorithm hinges critically upon the ability of the program to discriminate between points that lie within or without a region. The easiest probability distribution to handle is a rectangular distribution. For such a distribution, a typical algorithm for performing an integration over a region with a boundary defined in discrete format proceeds as follows:

(a) Border the region with a rectangle defined by the extreme coordinates in such a way that the region R is just contained within the rectangle.
(b) Generate pseudo-random points (x_i, y_i) contained in the rectangle.
(c) Retain all such points that are contained within R as well as the rectangle. This involves using OUTIN to test each generated point with respect for its location.
(d) Compute f_i for each successful point and add to the running sum $\Sigma_i f_i$.
(e) Repeat (b) through (d) until a preset number of N points contained in R is obtained.
(f) Approximate the value of the integral according to equation (14) above.

Results obtained by the author using this algorithm in the context of boundary elements may be found elsewhere.[5,6] Here, a simple example of finding some section properties of a multiply connected region is presented. We consider a circular diaphragm with an off-center circular



(a)



(b)

Figure 9. (a) A multiply-connected region defined by two circular boundaries. (b) The boundary mesh defined by node numbers sequenced as shown

inner boundary as depicted in Fig. 9(a). A 48-node boundary discretization of the region and the associated numbering scheme is shown in Fig. 9(b). As prescribed by the residue theorem, the internal surface is numbered in a clockwise manner opposite that of the external boundary. We desire the x-ordinate of the centroid and the area moments of inertia around the x and y axes. Working this problem involves integrating over the area $f(x, y)$ equal in succession to x, x^2, and y^2. The same distribution of spread of random points and the results of the calculations are shown in Fig. 10.

SUMMARY AND CONCLUSIONS

The residue theorem from complex variable theory has been used to construct a computer program with the capability of determining whether or not a point resides within a two-dimensional region. The region may be of arbitrary geometric complexity. The program has been shown to be useful in two applications of engineering interest. These are (1) the application of nonuniform boundary conditions in finite element analysis, and (2) Monte Carlo quadrature over an arbitrary region. It is concluded that the algorithm offers substantial time-saving advantages to an engineering analyst.

REFERENCES

1 Sloan, S. W. A point-in-polygon program, *Adv. Eng. Software* 1985, 7 (1), 45

2 Nordbeck, S. and Rystedt, B. Computer cartography point-in-polygon programs, *BIT* 0000, 7, 39

3 Churchill, R. V. *Complex Variables and Applications*, 2nd edn, McGraw-Hill, New York, 1960

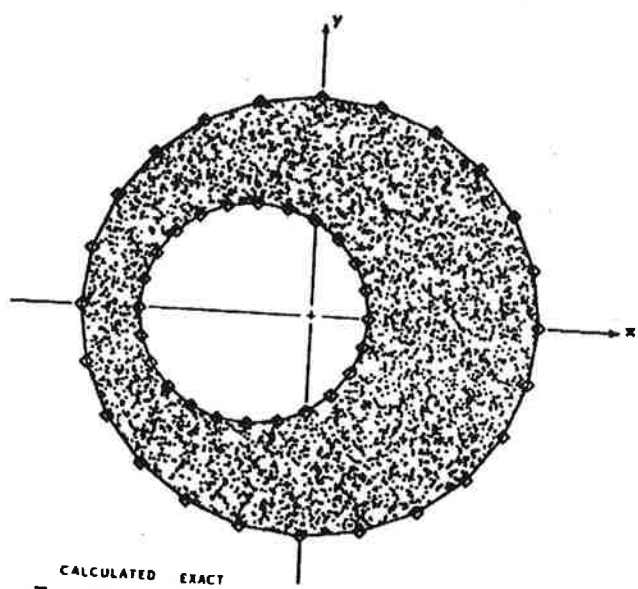
4 Brebbia, C. A., Telles, J. C. F. and Wrobel, L. C. *Boundary Element Techniques*, Springer-Verlag, Berlin, 1984

5 Gipson, G. S. Effective use of Monte Carlo quadrature for handling domain integrals in Poisson-type problems, *Engineering Analysis* 1985, 2 (3), 138

6 Gipson, G. S. *The Coupling of Monte Carlo Quadrature with the Boundary Integral Equation Technique to Solve Poisson Type Equations*, PhD Dissertation, Louisana State University, Baton Rouge, 1982

7 Kahn, H. Multiple quadrature of Monte Carlo methods, in Ralston, A. and Wilf, H. S. (eds.) *Mathematical Methods for Digital Computers*, John Wiley, 1960

8 Rubinstein, R. T. *Simulation and the Monte Carlo Method*, John Wiley, 1981



	CALCULATED	EXACT
$\bar{X}$	.084	.083
I_x	.730	.736
I_y	.676	.687

Figure 10. Random points distributed only within the shaded area of Fig. 9(a). The selection of the points was done with OUTIN and the boundary grid of Fig. 9(b).

```
      FUNCTION OUTIN(NUMNP,X,Y,XP,YP)
C
C     ==========================================================
C     =                                                        =
C     =  THIS FUNCTION DETERMINES WHETHER OR NOT A POINT       =
C     =  RESIDES WITHIN A TWO-DIMENSIONAL DOMAIN               =
C     =                                                        =
C     =  CALLING ARGUMENTS -                                   =
C     =                                                        =
C     =  NUMNP - NUMBER OF DEFINING BOUNDARY NODES             =
C     =                                                        =
C     =  X,Y - VECTORS CONTAINING BOUNDARY COORDINATE PAIRS IN =
C     =  CCW ORDER ON EXTERNAL SURFACE (CW OTHERWISE). THESE   =
C     =  ARE OF LENGTH NUMNP+1 WHERE X(NUMNP+1) = X(1), ETC.   =
C     =                                                        =
C     =  XP,YP - COORDINATES OF POINT IN QUESTION              =
C     =                                                        =
C     =                                                        =
C     =  VALUE RETURNED -                                      =
C     =                                                        =
C     =  OUTIN = 0.0 IF POINT IS NOT IN DOMAIN                 =
C     =        = 1.0 IF POINT IS INSIDE DOMAIN                 =
C     =        = 0.5 IF POINT LIES ON THE BOUNDARY             =
C     =                                                        =
C     ==========================================================
C
      DIMENSION X(1),Y(1)
C
      OUTIN=0.5
      SUM=0.0
C
      DO 10 I=1,NUMNP
C
C
C        ==============================================
C        =                                            =
C        =  COMPUTE COMPONENTS OF TWO VECTORS FORMED BY =
C        =  JOINING POINT IN QUESTION WITH ENDPOINTS OF =
C        =  BOUNDARY SEGMENTS                         =
C        =                                            =
C        ==============================================
C
         AX=X(I)-XP
         BX=X(I+1)-XP
         AY=Y(I)-YP
         BY=Y(I+1)-YP
C
C        ==============================================
C        =                                            =
C        =  FORM CROSS PRODUCT OF THE VECTORS TO      =
C        =  DETERMINE SIGN OF ANGULAR SEGMENT         =
C        =                                            =
C        ==============================================
C
         CP=AX*BY-AY*BX
         SGN=SIGN(1.0,CP)
```

AX=0
BX=0

SGN = ±1

```
C       ==========================================
C       =                                        =
C       =  TEST FOR VANISHING CROSS PRODUCT      =
C       =  IN CASE TEST POINT IS ON THE BOUNDARY =
C       =                                        =
C       ==========================================
C
          IF(ABS(CP).LT.1.E-06)RETURN
C
C       ==========================================
C       =                                        =
C       =  COMPUTE DOT PRODUCT OF TWO VECTORS FOR =
C       =  PURPOSE OF OBTAINING THE ACTUAL ANGLE =
C       =  SUBTENDED BY THE BOUNDARY INCREMENT   =
C       =                                        =
C       ==========================================
C
          DOT=AX*BX+AY*BY
          A2=AX*AX+AY*AY
          B2=BX*BX+BY*BY
          PROD=A2*B2
C
C       ==================
C       =                =
C       =  COMPUTE ANGLE =
C       =                =
C       ==================
C
          ANG=0.0
          IF(PROD.NE.0.0)ANG=ACOS(DOT/SQRT(PROD))
C
C       ============================
C       =                          =
C       =  ADD ANGLE TO RUNNING SUM =
C       =                          =
C       ============================
C
          SUM=SUM+ANG*SGN
C
   10   CONTINUE
C
C     ==========================================================
C     =                                                        =
C     =  CLEAN UP ROUNDOFF ERRORS IN SUM AND COMPUTE 'OUTIN'   =
C     =                                                        =
C     ==========================================================
C
        FCTR=SUM*0.159154943
        IF(ABS(1.0-FCTR).LT.1.E-02)OUTIN=1.0
        IF(ABS(FCTR).LE.1.E-02)OUTIN=0.0
C
        RETURN
        END
```